Sistemas Distribuidos (1959). Grupo: 2
Alumno: Rodríguez Jaramillo Alejandro
UNAM / FI / DIE
Semestre 2027-1

Bitácora S07 - Mexi Banco: de un monolito a microservicios

Paso 1: La historia

Al separar la aplicación monolítica en cinco microservicios independientes, se perdió la garantía transaccional local (`@Transactional`). Anteriormente, si una operación como el abono fallaba, la base de datos deshacía automáticamente los cambios porque todo residía en un único proceso y contexto de memoria. Al migrar a comunicación HTTP, cada servicio procesa y confirma únicamente su propia base de datos; si se producen fallos de red a mitad de una transacción compuesta, existe el riesgo de dejar el sistema en un estado inconsistente.

Paso 2: Directorio (Eureka)

En la configuración inicial, el tablero de Eureka en el puerto 8761 no respondía. Tras habilitar el servidor de descubrimiento y configurar los clientes en cada módulo, el tablero logró registrar exitosamente a los cinco servicios del sistema.

Respuesta "Piensa": Aunque los microservicios ya reportan su existencia al directorio, el servicio de `transferencia` no lo utiliza de forma directa para localizar al servicio de `cuenta`. Continúa empleando una dirección estática (`http://cuenta:8080`) definida en las variables de entorno del archivo compose, delegando la resolución de nombres al DNS interno de Docker en lugar de consultar la agenda dinámica de Eureka.

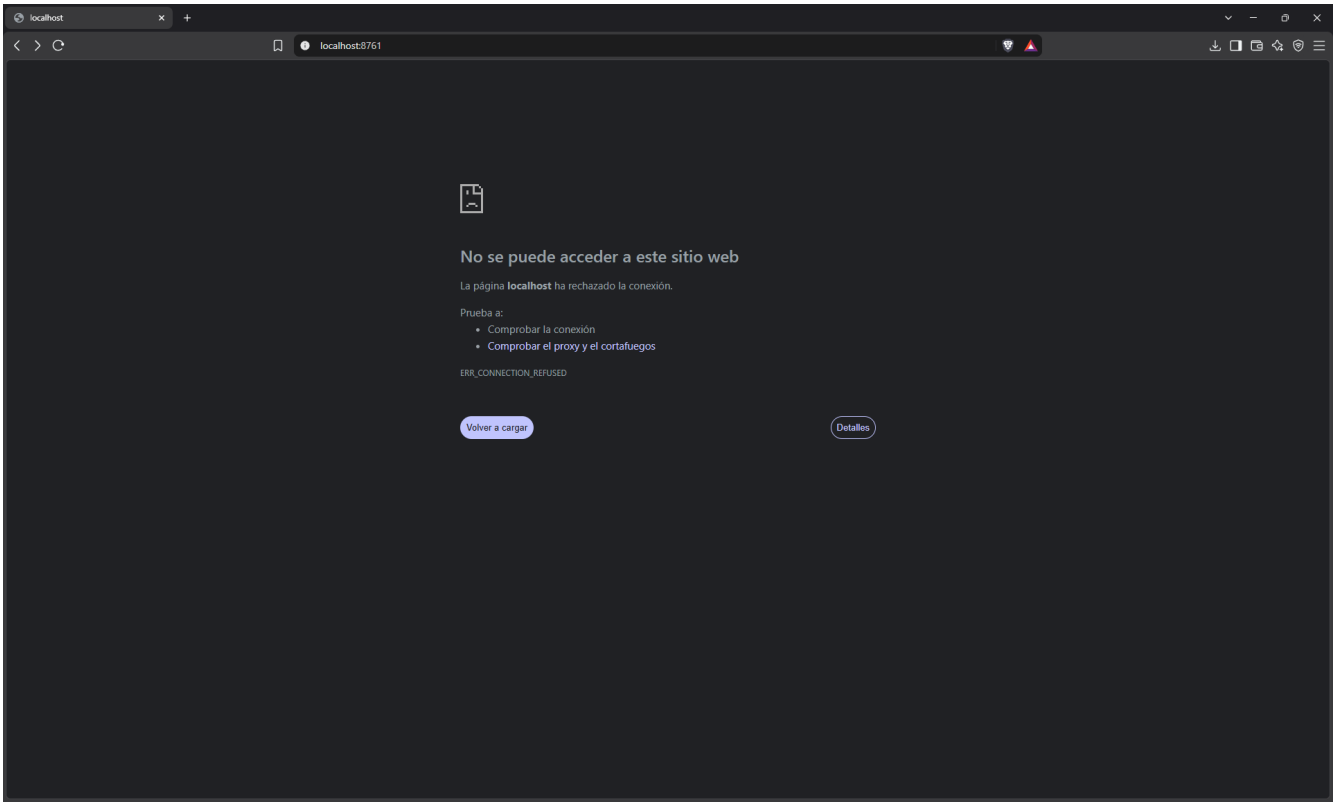


Figure 1: Tablero de Eureka antes de encender el servicio de descubrimiento.

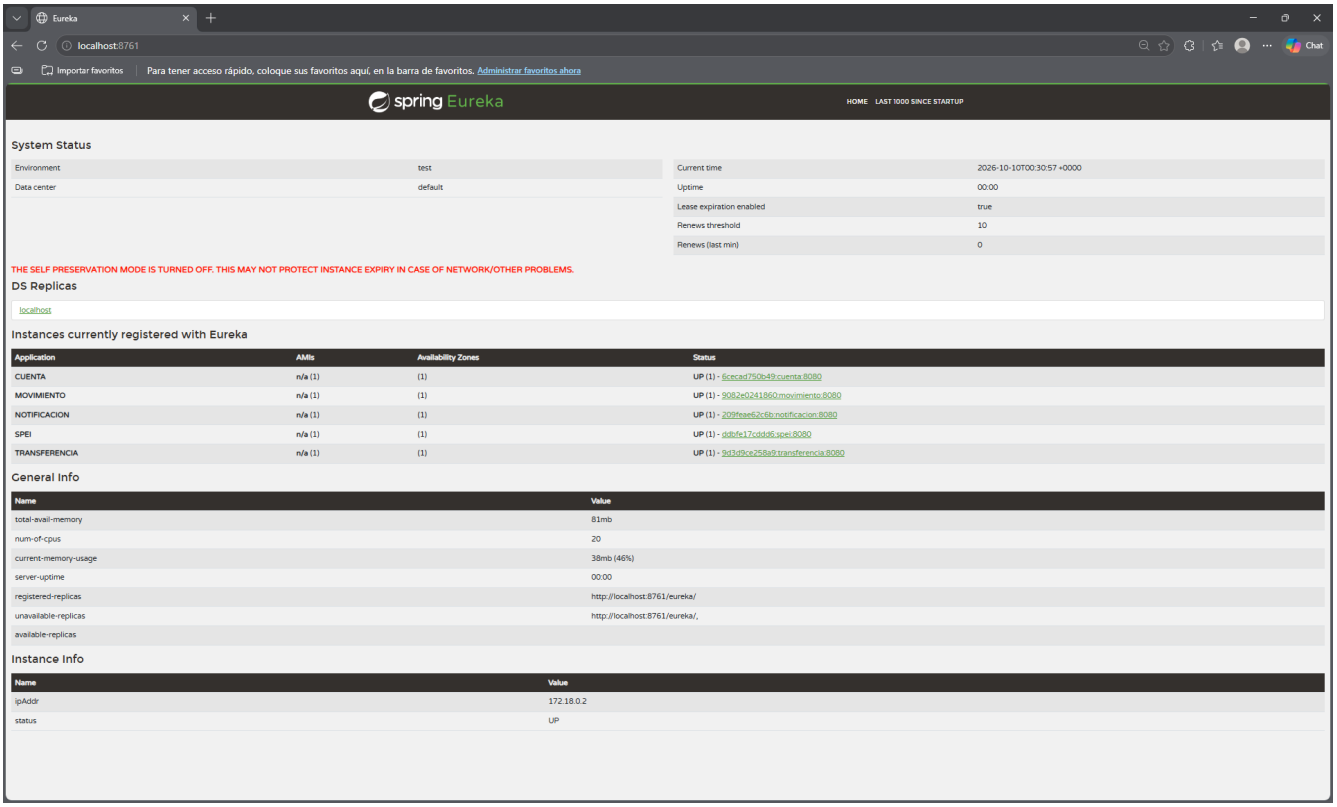


Figure 2: Tablero de Eureka mostrando los cinco servicios registrados correctamente y activos.

Paso 3: Gateway

El API Gateway funciona como la única puerta de entrada al sistema, centralizando las reglas de control de acceso. Las pruebas de acceso directo al puerto interno 8082 devolvieron un código de error 400. Al enrutar las peticiones a través del puerto 8080 del gateway, se obtuvieron dos respuestas con código 404 motivadas por razones distintas:

- El primer error 404 provino directamente del **servicio de cuenta** (evidenciado por la presencia de la cabecera `X-Instancia`), confirmando que la ruta y el microservicio existen, pero el número de cuenta consultado no fue hallado.
- El segundo error 404 fue emitido por el **Gateway**, el cual bloqueó e interrumpió la petición debido a que no existe una regla de enrutamiento pública expuesta para aceptar solicitudes POST hacia los endpoints internos de `movimientos`.

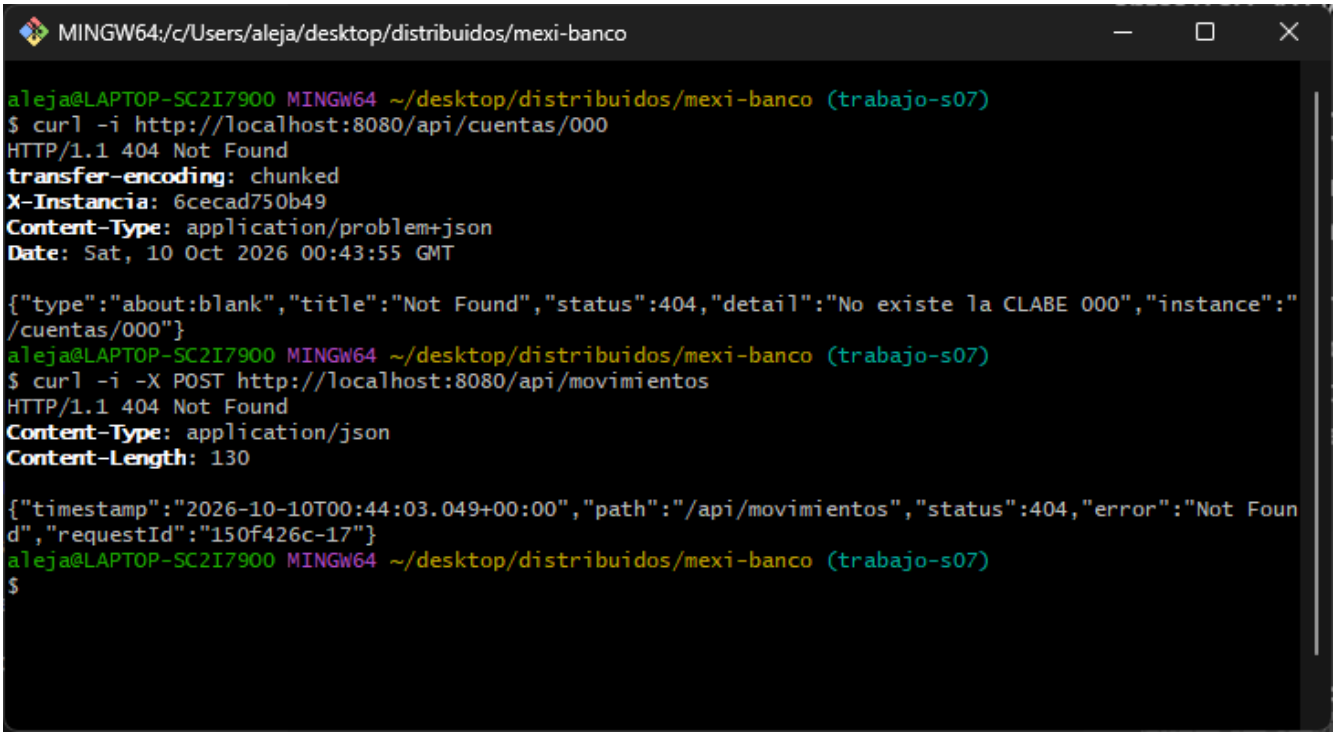


Figure 3: Prueba de errores 404 devueltos por el gateway y validaciones de seguridad.

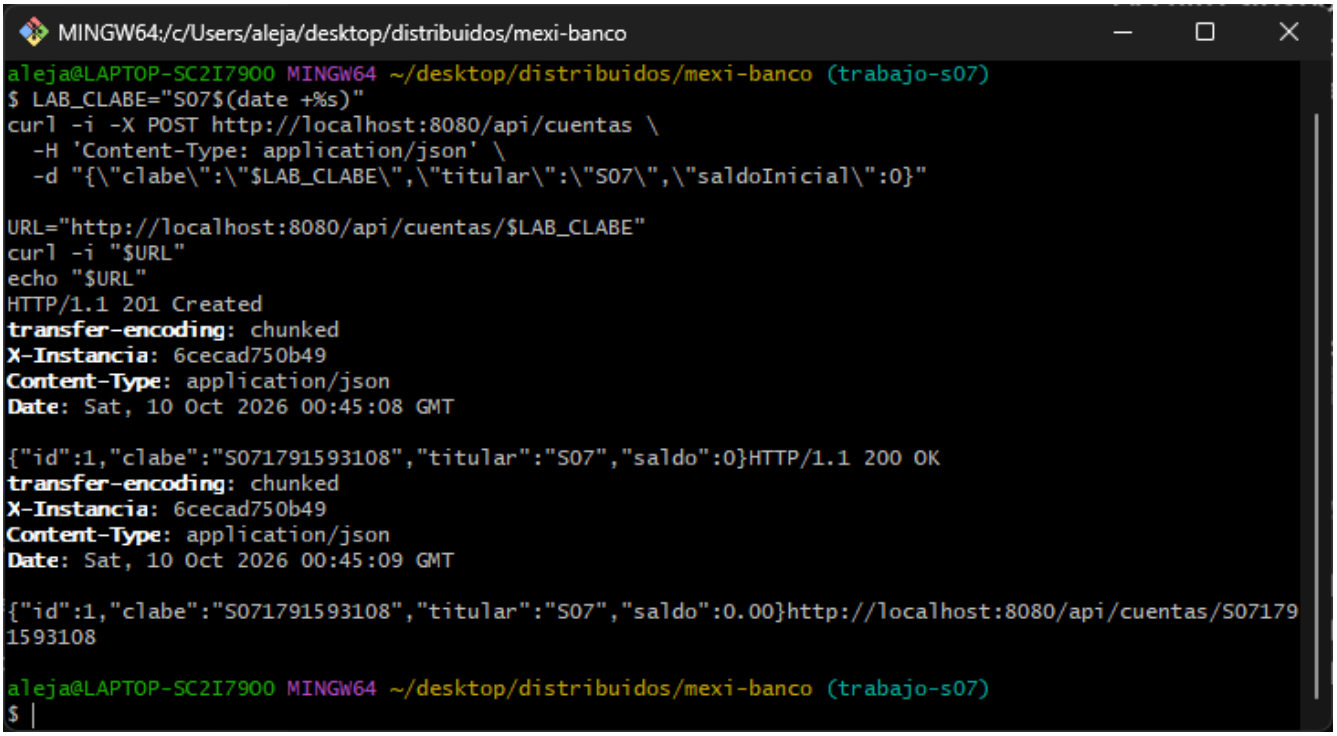


Figure 4: Validación de registro exitoso (201 Created) y consulta de saldo (200 OK) a través de la puerta única.

Pasos 4 y 5: Copias y caída

Tras escalar horizontalmente el microservicio de cuentas, el panel de administración reflejó **CUENTA UP (3)**. Al emitir peticiones simultáneas, las cabeceras `X-Instancia` demostraron que el sistema distribuyó la carga mediante un algoritmo *Round Robin* equitativo entre las tres réplicas disponibles.

Simulación de caída de nodo: A las 19:15:13 se interrumpió abruptamente el contenedor de la primera instancia de cuenta (‘docker kill‘). Durante el intervalo posterior, se registraron de forma intermitente códigos de error **500 y 503** en las peticiones concurrentes hasta las 19:15:36, momento en el cual el tráfico se estabilizó al 100% de éxito.

¿Por qué ocurrieron estos errores? Este comportamiento se debe al desfase temporal inherente en la propagación de eventos distribuidos. El directorio Eureka requiere el vencimiento del plazo de latidos (*heartbeats*) configurado en 15 segundos para declarar inactiva a una instancia, y de forma paralela el gateway opera con una caché interna de rutas. Durante esa ventana temporal de aproximadamente 23 segundos, el balanceador continuó enviando llamadas hacia una dirección de contenedor ya inexistente.

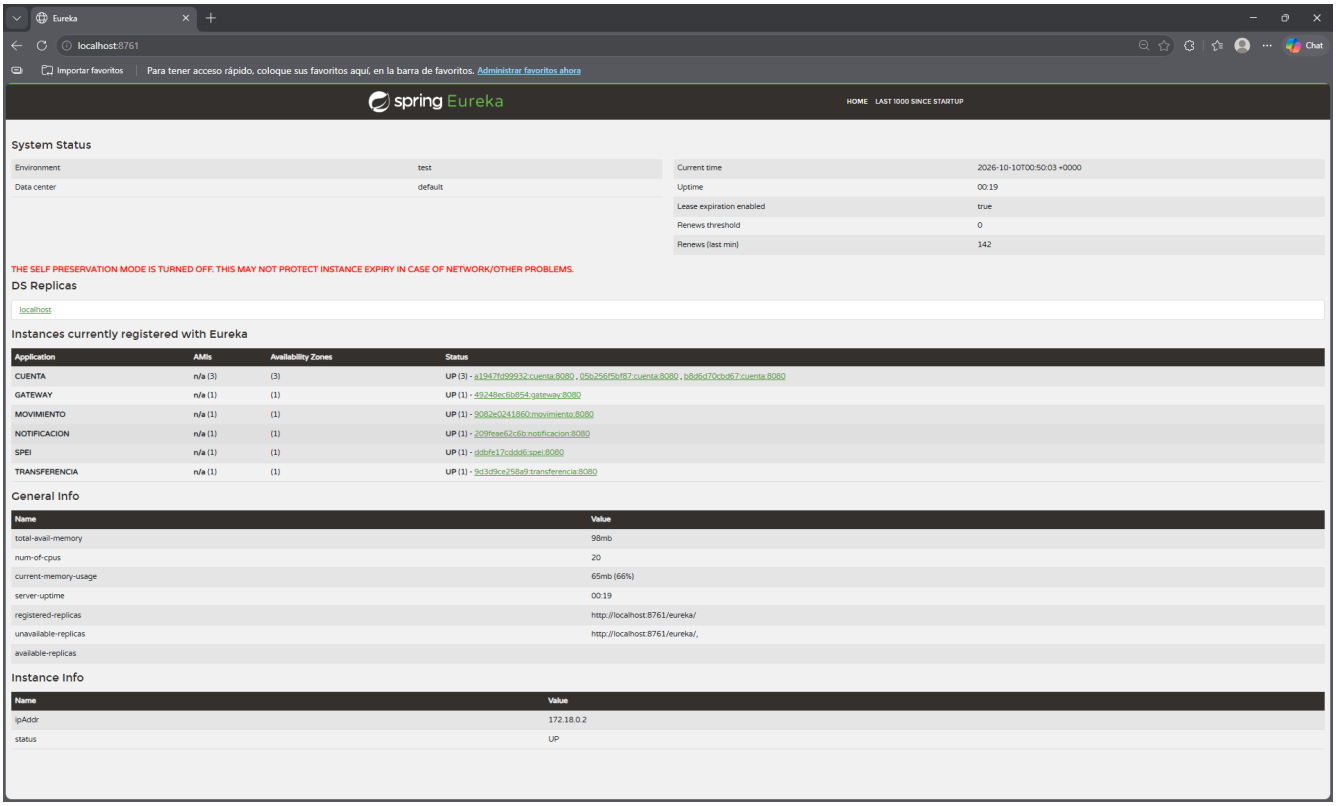


Figure 5: Panel de Eureka mostrando el escalamiento a tres instancias activas de CUENTA.

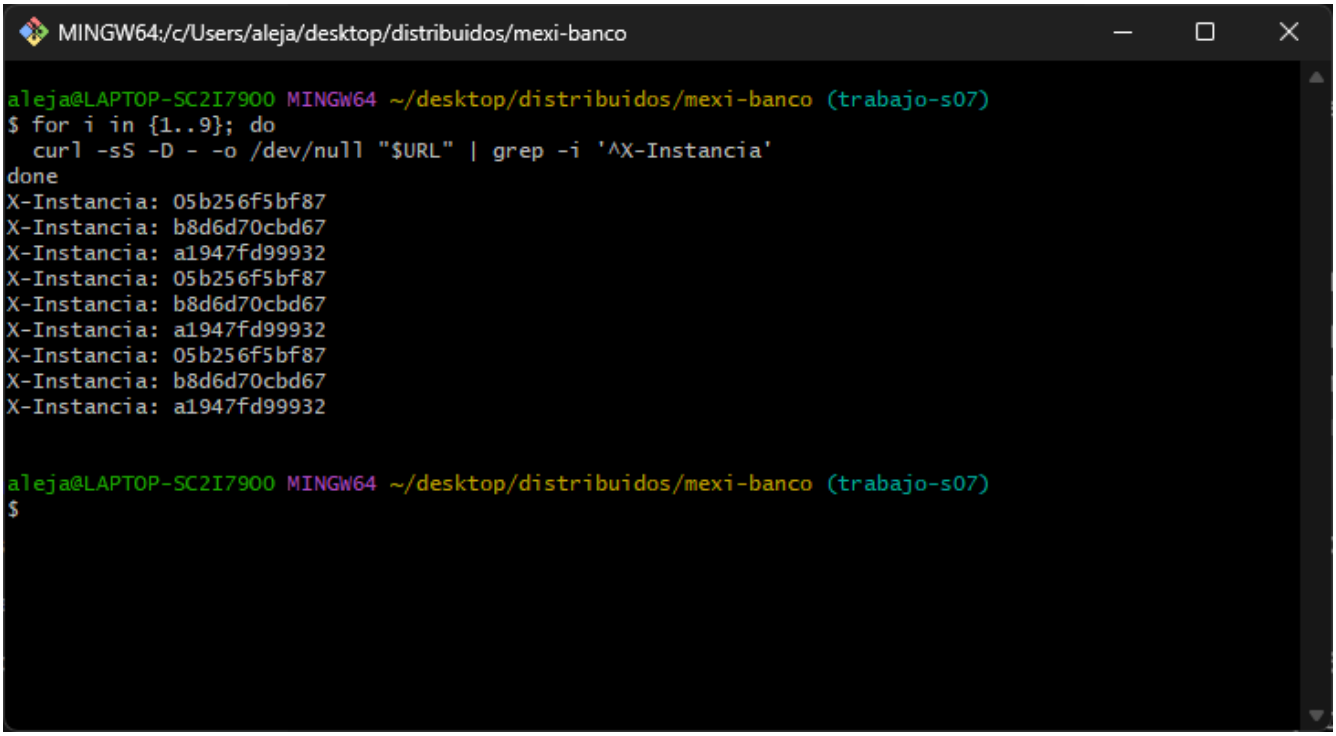


Figure 6: Validación del balanceo de carga por turnos mediante la cabecera `X-Instancia`.

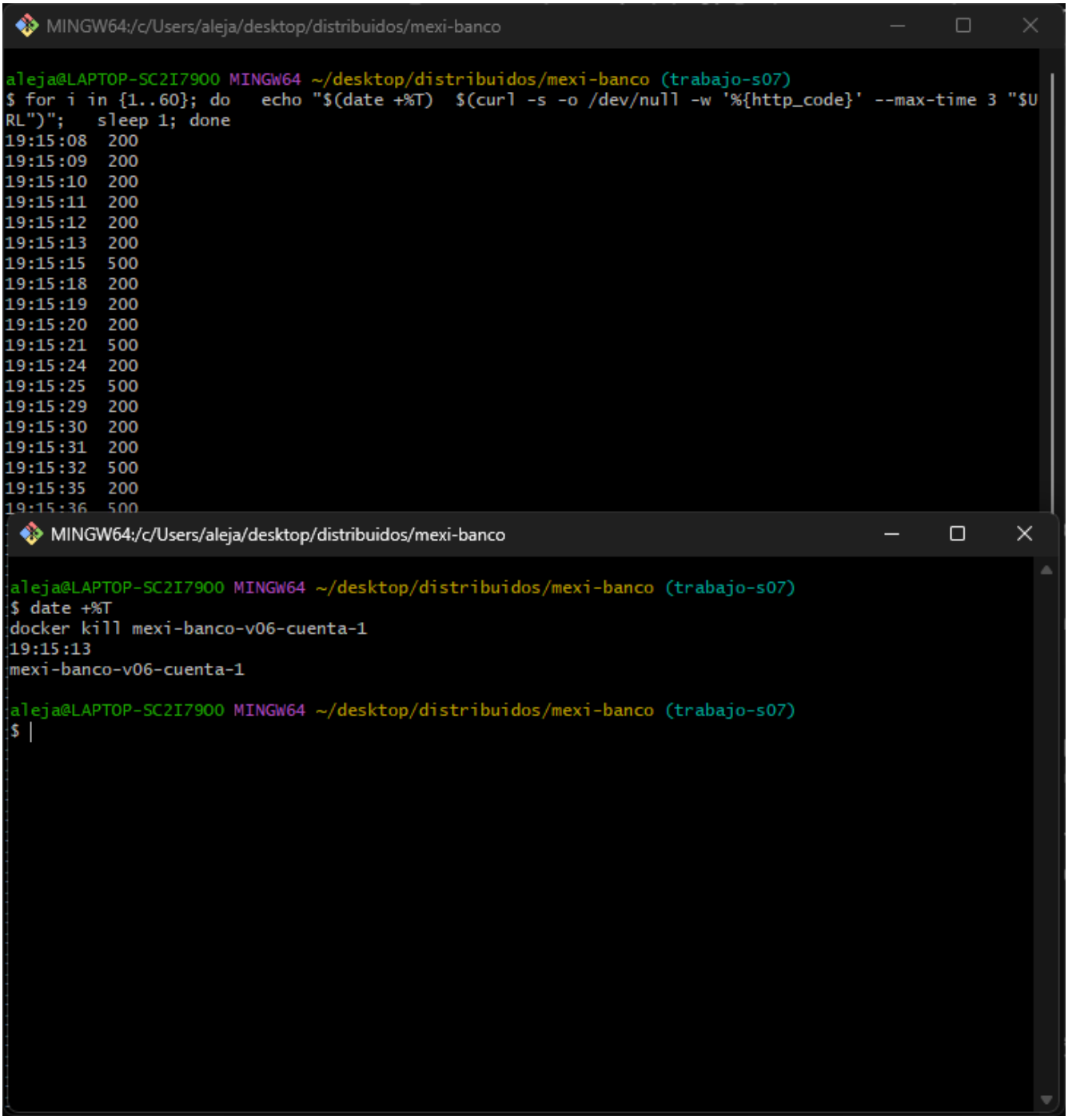


Figure 7: Monitoreo en tiempo real de códigos HTTP mostrando intermitencias de error tras la caída del nodo.

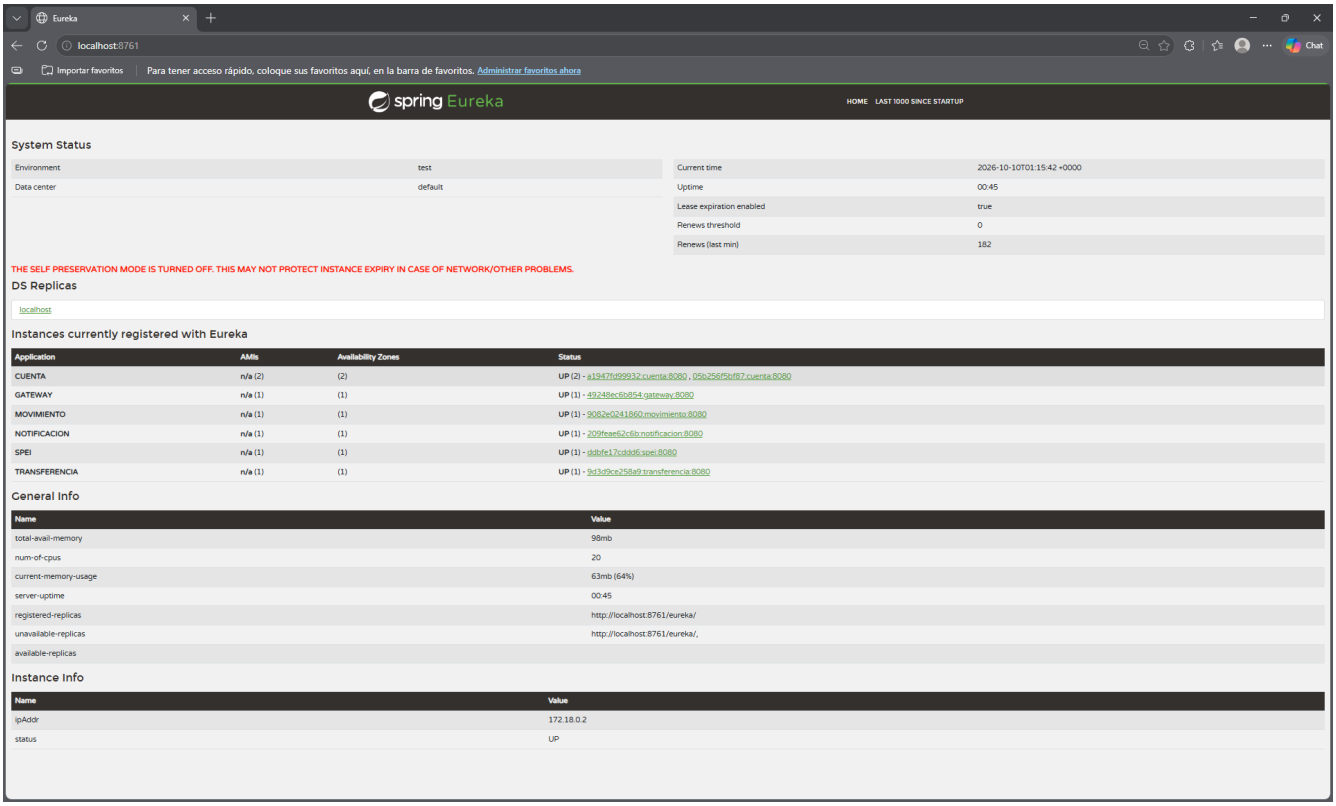


Figure 8: Registro de la hora exacta del comando de terminación y recuperación de estabilidad en el sistema.

Cierre

- El **directorio** resuelve la localización dinámica de servicios activos y sus direcciones IP, erradicando el acoplamiento estricto de direcciones estáticas.
- El **gateway** resuelve la consolidación de un punto único de acceso perimetral, resguardando y aislando los componentes lógicos internos de la red exterior.
- El **balanceador** resuelve la distribución equitativa de peticiones entre múltiples réplicas y tolera fallos mitigando el envío de tráfico a nodos caídos.
- **Durante la caída de una réplica se observaron** errores intermitentes 500/503 por espacio de 23 segundos debido al tiempo de expiración de los latidos en el directorio y la vigencia de la caché del gateway.

Tarea: Transferencia balanceada

Se configuró el cliente interno de transferencias implementando la anotación `@LoadBalanced` en el contexto de Spring. De este modo, el microservicio de transferencia consulta de manera dinámica al directorio

Eureka para resolver el nombre lógico `http://cuenta`. La ejecución exitosa del script de demostración demostró que las distintas copias atendieron de forma equitativa las operaciones de cargos y abonos.

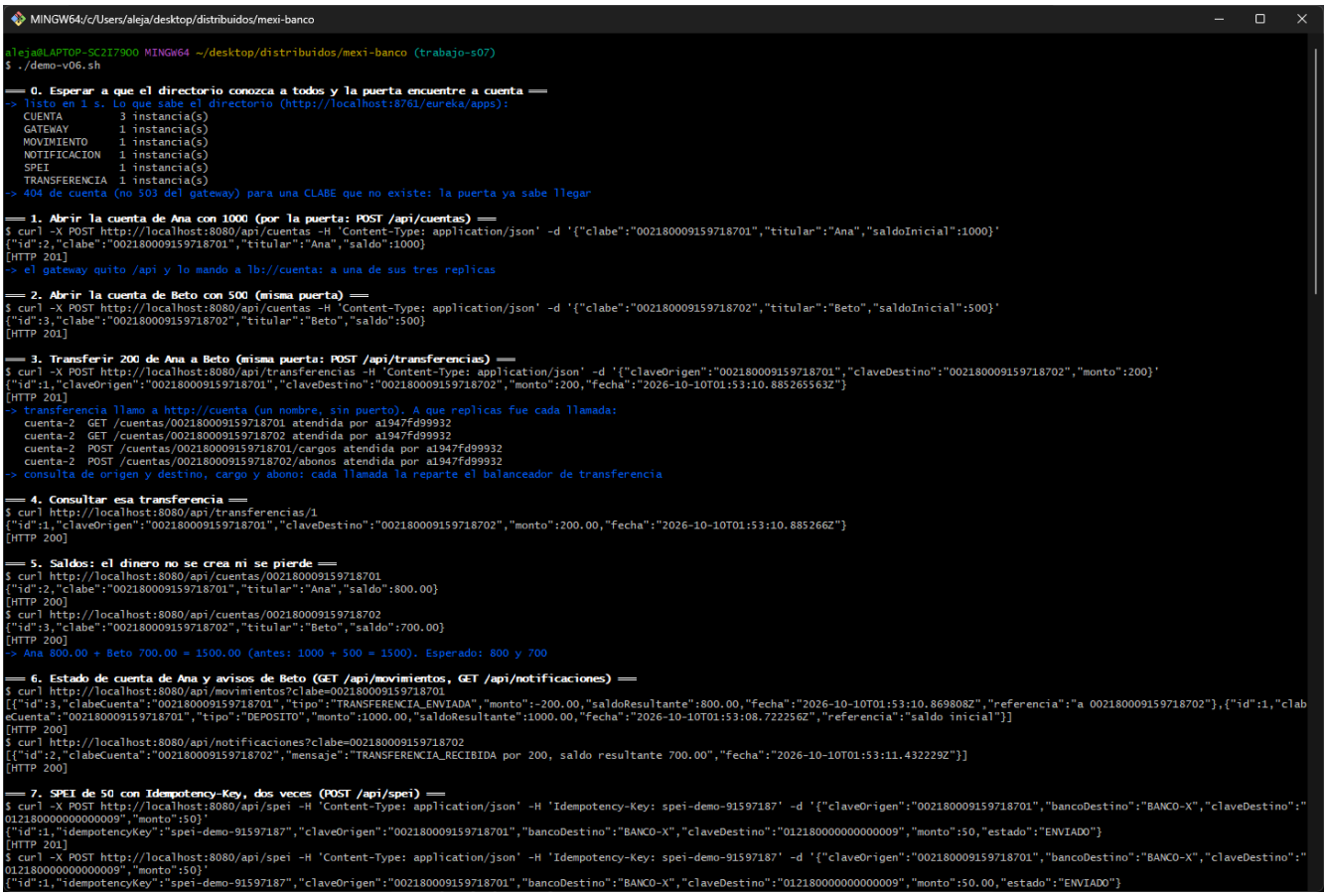


Figure 9: Inicialización y verificación de registros en el directorio durante la ejecución de la demo.

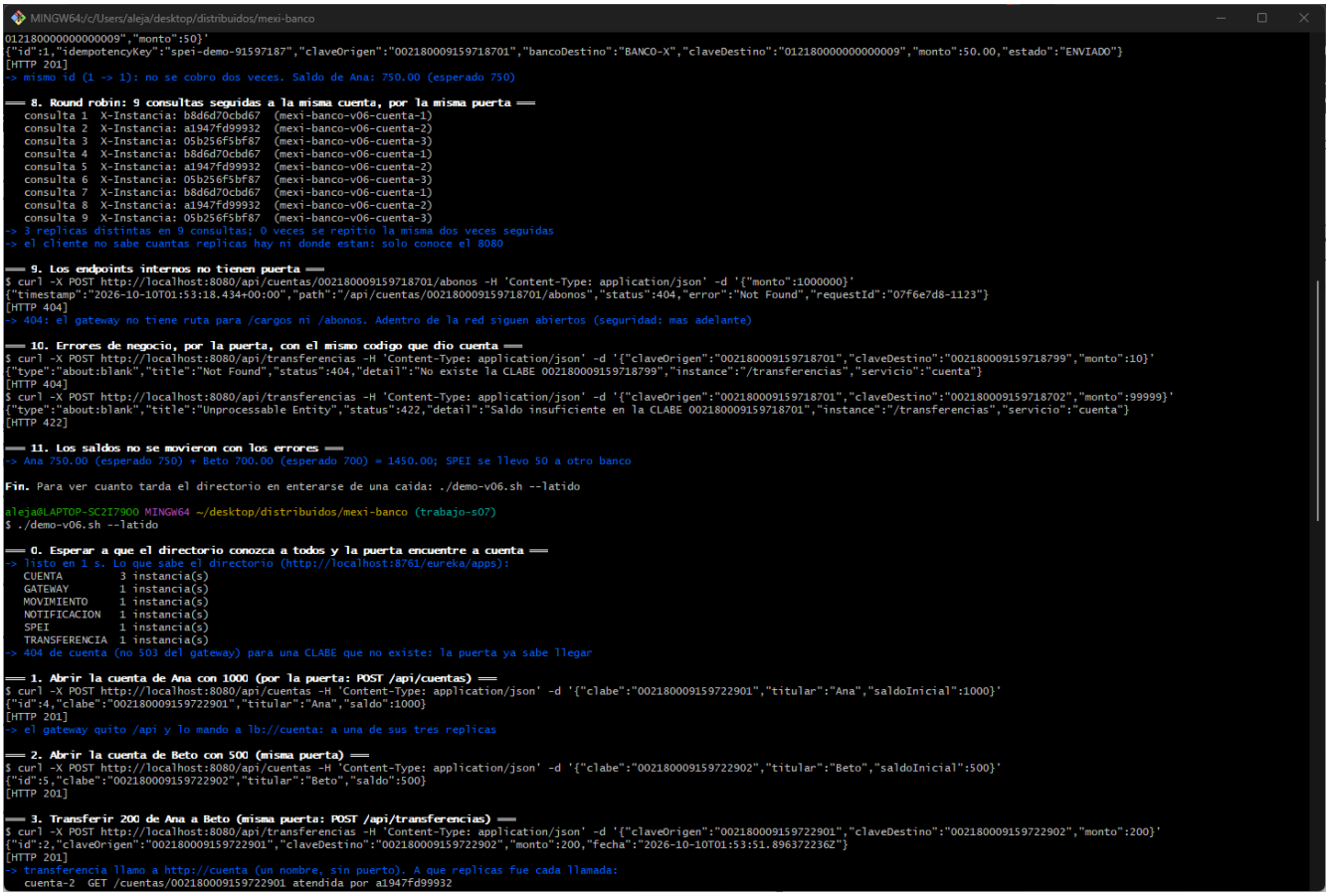


Figure 10: Creación de cuentas, procesamiento de transferencias y distribución de carga interna entre réplicas.

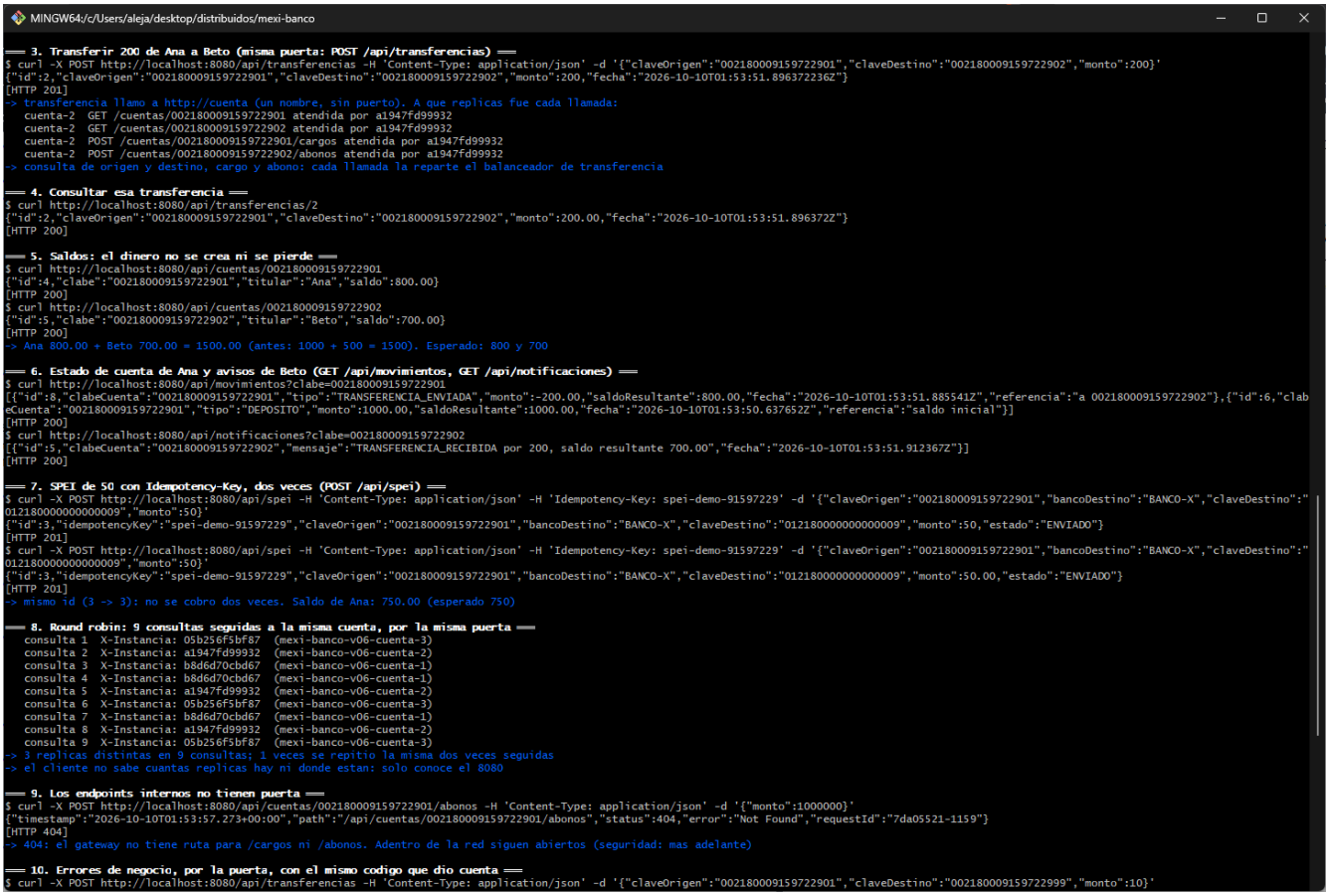


Figure 11: Validación de consistencia en saldos finales, operaciones SPEI y reglas de negocio.

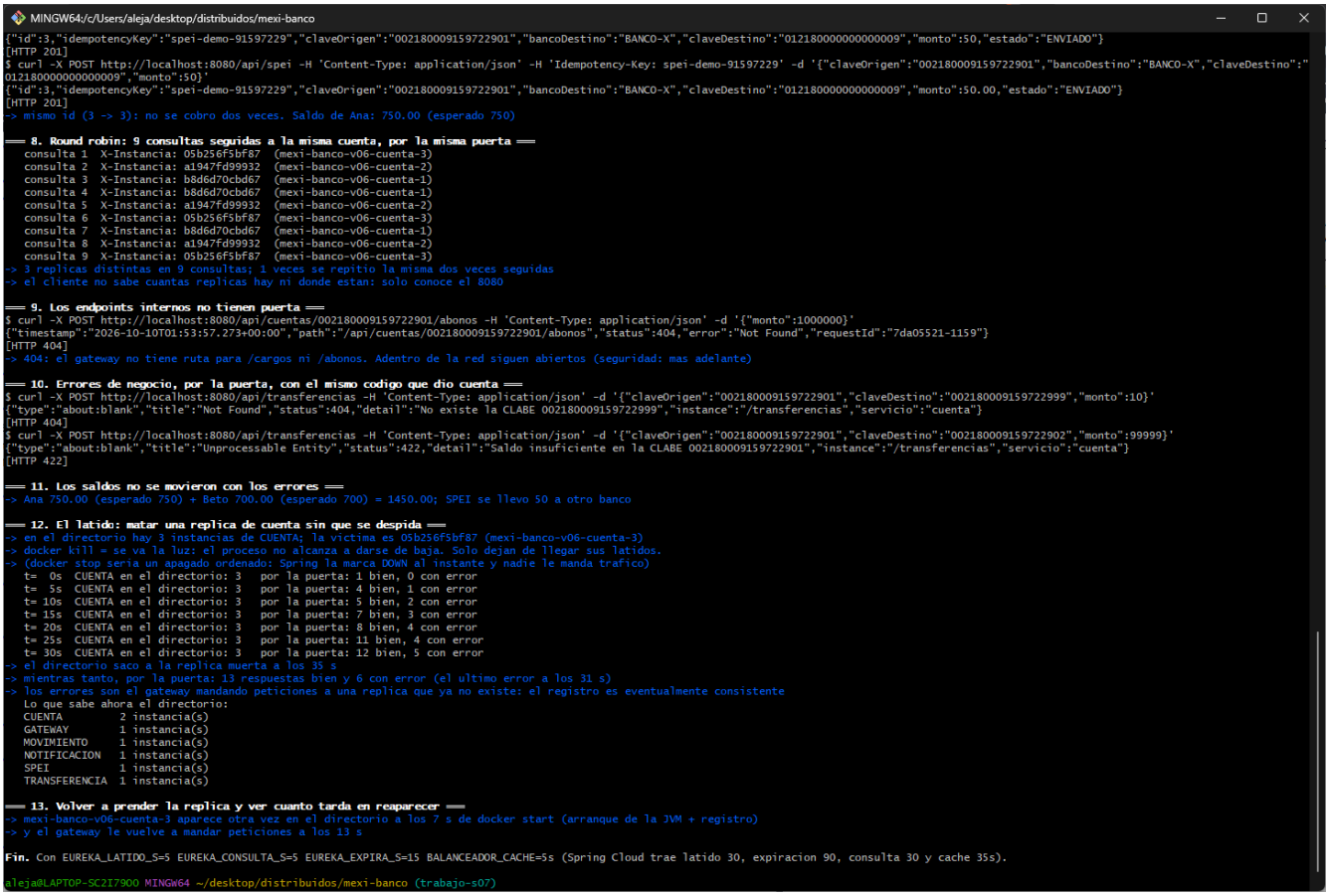


Figure 12: Resultado final de la ejecución de `./demo-v06.sh` comprobando el éxito operativo y el balanceo interno.

Correcciones Aplicadas

Los siguientes puntos y adiciones fueron añadidos a la bitácora para cumplir con lo solicitado en la evaluación de la Sesión 07.

1. Datos de la Instancia en el Servidor de Registro (XML Eureka)

Durante la configuración del Directorio de Servicios en la **Parte 2**, se consultó la dirección web del servicio de cuentas para verificar su registro. El resultado muestra los datos principales de la instancia guardada en el servidor Eureka, confirmando que la identificación y los tiempos de operación se configuraron correctamente:

```xml
<application>
  <name>CUENTA</name>
  <instance>
    <instanceId>6bd55a1c1cd7:cuenta:8080</instanceId>
    <hostName>172.18.0.2</hostName>
    <app>CUENTA</app>
    <ipAddr>172.18.0.2</ipAddr>
    <status>UP</status>
    <port enabled="true">8080</port>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
    </leaseInfo>
  </instance>
</application>
```

Listing 1: Respuesta XML al consultar la ruta /eureka/apps/CUENTA en el directorio

2. Predicción sobre la Caída de un Nodo

Antes de simular que una de las copias del servicio fallaba en la **Parte 5**, se planteó la siguiente idea basada en cómo funciona el reparto de trabajo por turnos (*round-robin*):

> *El sistema mostrará errores de manera intermitente, afectando únicamente a una de cada tres peticiones que lleguen al mismo tiempo. Estos fallos durarán solo el tiempo necesario para que el directorio note la falta de señales de vida y actualice su lista de servicios activos. Una vez que la copia apagada sea retirada del registro, las peticiones volverán a responder sin problemas.*

3. Registro y Medición de la Caída

Las pruebas realizadas en la práctica comprobaron la predicción anterior mediante el registro de los tiempos exactos obtenidos en la terminal:

- **Apagado del servicio:** El contenedor principal `mexi-banco-v06-cuenta-1` fue detenido a las **19:15:13**.
- **Recuperación total:** Después de un tiempo con errores intermitentes en las consultas, el sistema volvió a responder de forma correcta y estable a las **19:15:36**.
- **Aviso en el Directorio:** Al mismo tiempo, se revisó el panel de control de Eureka, donde el estado general del servicio cambió de **UP(3) a UP(2)** a las **19:15:33**.

Estos datos demuestran que hubo un retraso de **20 a 23 segundos**. Este tiempo se debe a la suma del plazo que espera el directorio para dar de baja un servicio que no responde (15 segundos) más el tiempo que tarda la puerta de enlace (*gateway*) en actualizar su lista interna de rutas (5 segundos).